

Лекция №7. Функции

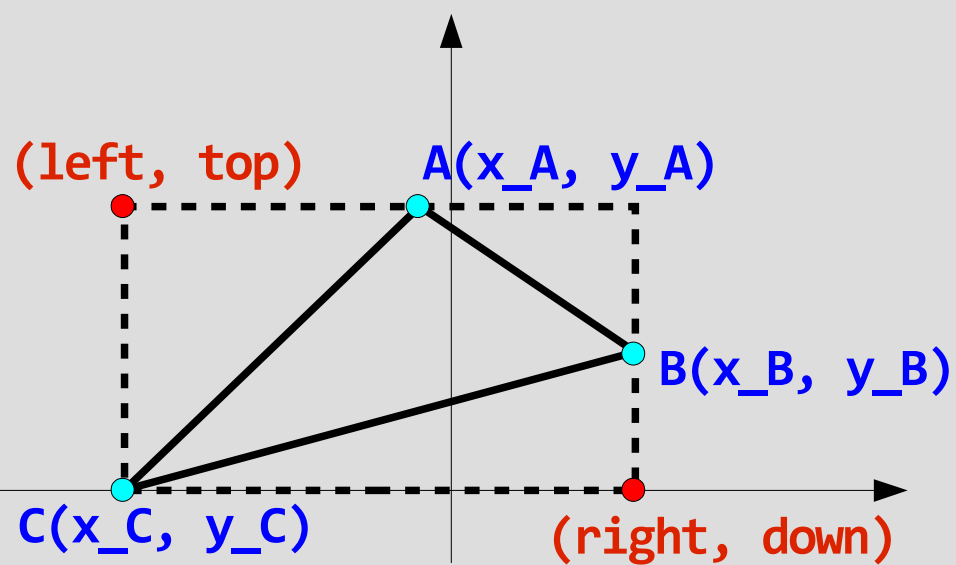
- Понятие функции в языке Си
- Объявление и описание функции
- Вызов функции
- Передача данных в функцию и возвращение данных из функции

Функция

Функция - это **именованная последовательность объявлений и операторов**, выполняющая какое-либо законченное действие. Функция может принимать и возвращать значения

Пример программы, состоящей из единственной функции

Для заданного треугольника **ABC** определить прямоугольную область, которая его описывает



```
void main()
```

Пример программы, состоящей из единственной функции

```
int _tmain(int argc, _TCHAR* argv[])
{
    int    x_A = 5, y_A = 4, x_B = -6,
           y_B = 6, x_C = 3, y_C = 0; // вершины треугольника
    int    left, top, right, down;    /* границы прямоугольной
                                       области, содержащей треугольник */
}
```

```
// Находим левую границу: left = min(x_A, x_B, x_C)
if      (x_A <= x_B && x_A <= x_C)    left = x_A;
else if (x_B <= x_A && x_B <= x_C)    left = x_B;
else                                     left = x_C;
```

```
// Находим нижнюю границу: down = min(y_A, y_B, y_C)
if      (y_A <= y_B && y_A <= y_C)    down = y_A;
else if (y_B <= y_A && y_B <= y_C)    down = y_B;
else                                     down = y_C;
```

Пример программы, состоящей из единственной функции

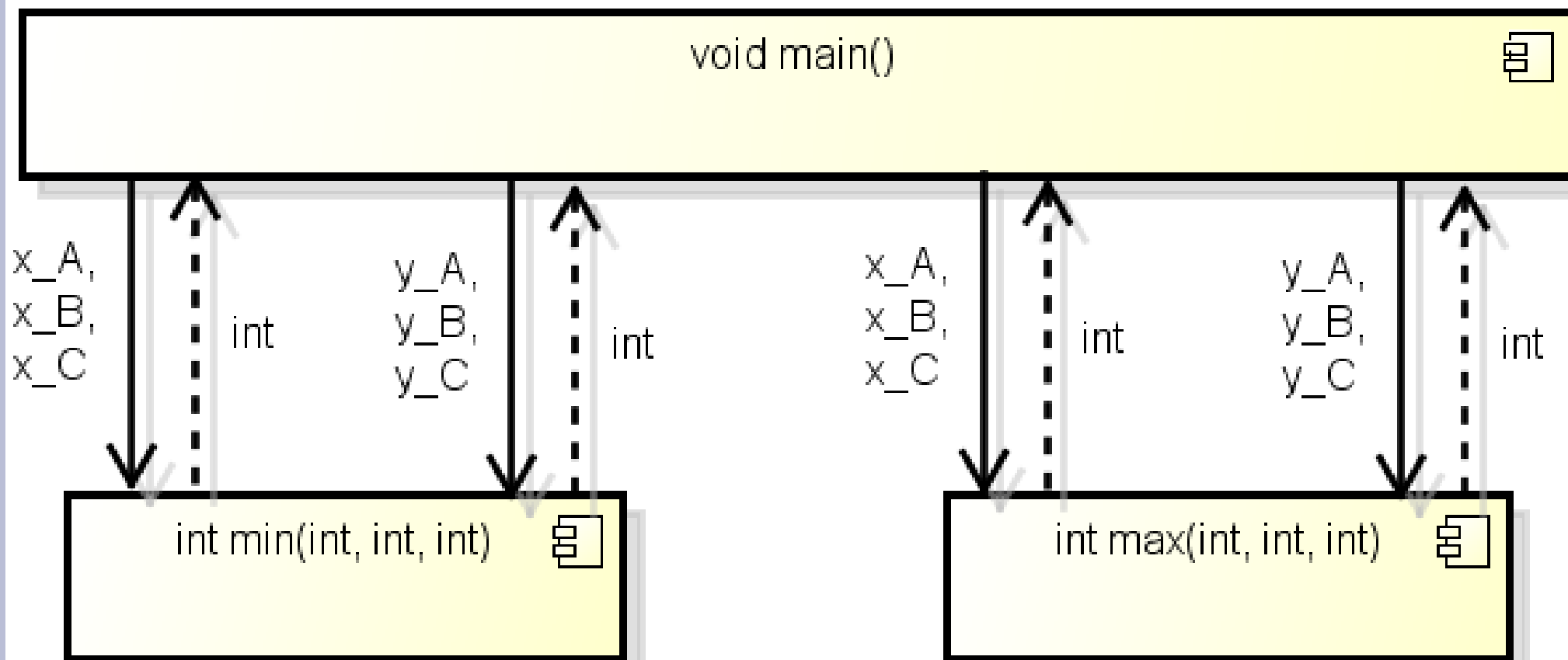
```
// Находим правую границу: right = max(x_A, x_B, x_C)
if      (x_A >= x_B && x_A >= x_C)    right = x_A;
else if (x_B >= x_A && x_B >= x_C)    right = x_B;
else                                     right = x_C;
```

```
// Находим верхнюю границу: top = max(y_A, y_B, y_C)
if      (y_A >= y_B && y_A >= y_C)    top = y_A;
else if (y_B >= y_A && y_B >= y_C)    top = y_B;
else                                     top = y_C;
```

```
printf("Area = (%d, %d) - (%d,%d)", left, top, right, down);
return 0;
```

```
}
```

Пример программы, состоящей из нескольких функций



Пример программы, состоящей из нескольких функций

```
int min(int v1, int v2, int v3) // Поиск минимума трех значений
{
    int result;

    if      (v1 <= v2 && v1 <= v3) result = v1;
    else if (v2 <= v1 && v2 <= v3) result = v2;
    else                                     result = v3;

    return result;
}
```

```
int max(int v1, int v2, int v3) // Поиск максимума трех значений
{
    int result;

    if      (v1 >= v2 && v1 >= v3) result = v1;
    else if (v2 >= v1 && v2 >= v3) result = v2;
    else                                     result = v3;

    return result;
}
```

Пример программы, состоящей из нескольких функций

```
int _tmain(int argc, _TCHAR* argv[])
{
    int    x_A = 5, y_A = 4, x_B = -6,
           y_B = 6, x_C = 3, y_C = 0; // вершины треугольника
    int    left, top, right, down;    /* границы прямоугольной
                                     области, содержащей треугольник */

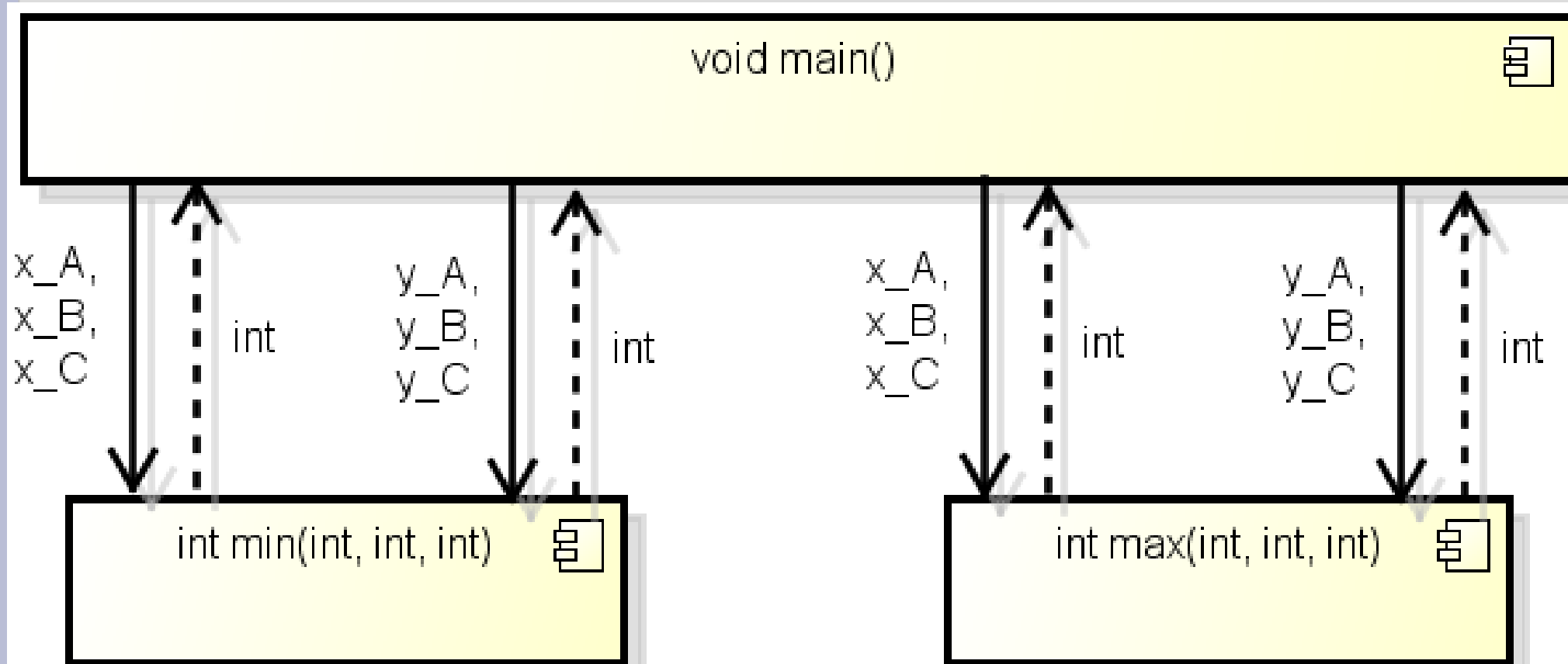
    left =  min(x_A, x_B, x_C);      // Находим левую границу
    down =  min(y_A, y_B, y_C);      // Находим нижнюю границу
    right = max(x_A, x_B, x_C);      // Находим правую границу
    top =   max(y_A, y_B, y_C);      // Находим верхнюю границу

    printf("Area = (%d, %d) - (%d,%d)", left, top, right, down);
    return 0;
}
```


Автономность / зависимость функций

- Функция – это **обособленный** участок кода со **своим** набором переменных и операций, к которым другие функции **не имеют** непосредственного доступа
- Программа представляет собой **набор автономных** функций, которые **взаимодействуют** между собой посредством **вызова** друг друга
- При вызове функции **обмениваются** между собой данными — в функцию поступают **входные** данные, а из функции — **выходные**

Автономность / зависимость функций



Описание функции

- Если мы хотим создать собственную функцию, то мы должны задать ее **описание**:

```
<заголовок функции>  
{  
    <тело функции>  
}
```

- **Заголовок** сообщает компилятору **имя** функции и ее входные/выходные **данные**
- **Тело** функции определяет последовательность **операторов**, реализующих назначение функции

Заголовок функции

- **Заголовок** функции определяет ее имя, набор параметров и возвращаемое значение:

<тип возвращаемого значения>
<имя функции> ([<параметр 1>, <параметр 2>, ...])

- Каждый <параметр> задается в виде <тип> <имя>
- **Пример:**

```
double pow( double x, double y )
```

- При **отсутствии** возвращаемого значения указывается ключевое слово **void** (пустота)

Задание

Составьте заголовок функции `min`, определяющей минимальное из двух значений

Входные данные:

- целое число
- целое число

Выходные данные:

- минимальное значение из двух

Заголовок функции

```
/*!  
 *   Поиск минимума двух значений  
 *   \param [in] val1 – первое значение  
 *   \param [in] val2 – второе значение  
 *   \return – минимум из val1 и val2  
 */  
int min( int val1, int val2 )
```

Тело функции

- Состоит из **объявления** локальных переменных и операторов
- Параметры, указанные в заголовке функции (называются **формальными**), могут использоваться в теле функции как **локальные** переменные
- Если функция **возвращает** значение, то должен присутствовать оператор **return**, который завершает выполнение функции и возвращает управление вызывающей функции

Оператор возврата из функции - `return`

`return` <выражение>;

- <выражение> должно быть **согласовано** по типу с возвращаемым значением функции. Собственно, значение выражения и является возвращаемым значением функции
- Если функция не имеет возвращаемого значения, то оператор `return` не обязателен. Если он используется, то выражение не указывается

Задание

Опишите тело функции `min`, определяющей минимальное из двух значений

Входные данные:

- целое число
- целое число

Выходные данные:

- минимальное значение из двух

Описание функции

```
/* Поиск минимума двух значений */
int min( int val1, int val2 )
{
    int result;    // результат

    if( val1 < val2 ) result = val1;
    else               result = val2;

    return result;    // возвращаем результат
}
```

```
/* Поиск минимума двух значений */
int min2( int val1, int val2 )
{
    return (val1 < val2) ? val1 : val2;
}
```

Вызов функции

`<имя функции> ( [<параметр 1>, <параметр 2>, ...] )`

- Вызов функции является операцией:
 - **результат** — возвращаемое значение; в отличие от других операций может отсутствовать
 - **операнды** — **фактические** параметры; в их роли могут выступать **любые выражения**, результат которых соответствует типу параметров
 - возможен **побочный эффект**
 - **приоритет операции** — один из самых высоких

Вызов функции

`<имя функции> ( [<параметр 1>, <параметр 2>, ...] )`

- **Признаком** операции является имя функции и круглые скобки, поэтому круглые скобки **обязательны**, даже при отсутствии параметров
- Для вызова функции достаточно знать **заголовок** функции

Задание

- Используя функцию

```
/*!  
 *    Поиск минимума двух значений  
 *    \param [in] val1 – первое значение  
 *    \param [in] val2 – второе значение  
 *    \return – минимум из val1 и val2  
 */  
int min( int val1, int val2 )
```

возведите в квадрат минимальное из двух чисел

```
int a, b;
```

Вызов функции

```
int result = min(a, b) * min(a, b);
```

Задание

- Используя функцию

```
/*!  
 *    Поиск минимума двух значений  
 *    \param [in] val1 – первое значение  
 *    \param [in] val2 – второе значение  
 *    \return – минимум из val1 и val2  
 */  
int min( int val1, int val2 )
```

найдите минимальное значение из трех и запишите в переменную **a**. Действие выполните в главной функции, а результат напечатайте на экране

```
int a, b, c;
```

Вызов функции

```
#include <stdio.h>

void main()
{
    int a, b, c;

    //Получение значений a, b и c

    a = min( min(a, b), c );

    printf("a = %d", a);
}
```


Структура программы, состоящей из нескольких функций

- Программа на языке Си состоит из директив препроцессора и описаний функций:

<подключение заголовочных файлов>

<описания функций>

<описание главной функции main>

- При этом необходимо учитывать, что описание функции должно располагаться раньше чем ее вызов

Пример программы, состоящей из нескольких функций

```
#include <stdio.h> // содержит прототип функции printf

int min2(int val1, int val2) /* Поиск минимума двух
                             значений */
{
    return (val1 < val2) ? val1 : val2;
}

int min3(int val1, int val2, int val3)/* Поиск минимума
                                       трех значений */
{
    return min( min(val1, val2), val3);
}

void main() /* Главная функция */
{
    int a = 6, b = 3, c = 7;
    printf("a = %d", min3(a, b, c) );
}
```

Прототип функции

- Чтобы не следить за порядком описания функций используют их **предварительные объявления** или **прототипы** функций
- Задание **прототипа** функции в **начале** программы позволяет описывать функцию **после** ее вызовов
- Прототип **похож** заголовку со следующими отличиями:
 - имена аргументов указывать **не** обязательно;
 - в конце ставится **точка с запятой**

<заголовок>;

Задание

Напишите прототип функции `min`, определяющей минимальное из двух значений

Входные данные:

- целое число
- целое число

Выходные данные:

- минимальное значение из двух

Прототип функции

```
int min( int val1, int val2 );
```

```
int min( int , int );
```

Структура программы с использованием прототипов функций

- Программа на языке Си состоит из директив препроцессора, прототипов функций и описаний функций:

<подключение заголовочных файлов>

<прототипы функций>

<описания функций>

- Прототипы и описания функций нельзя располагать внутри других функций
- Прототип функции `main` задавать не требуется

Задание

Напишите программу, печатающую минимальное значение из трех заданных

Используйте прототипы. **Выполните** описание главной функции раньше остальных функций

Пример программы, состоящей из нескольких функций

```
#include <stdio.h> // содержит прототип функции printf

int min2(int, int );    // Поиск минимума двух значений
int min3(int, int, int); // Поиск минимума трех значений

// Функции main и min могут быть заданы в любом порядке
// т. к. описаны прототипы
void main()              // Главная функция
{
    int a = 6, b = 3, c = 7;
    printf("a = %d", min3(a, b, c) );
}

int min3(int val1, int val2, int val3)
{ return min( min(val1, val2), val3); }

int min2(int val1, int val2)
{ return (val1 < val2) ? val1 : val2; }
```


Передача параметров в функцию

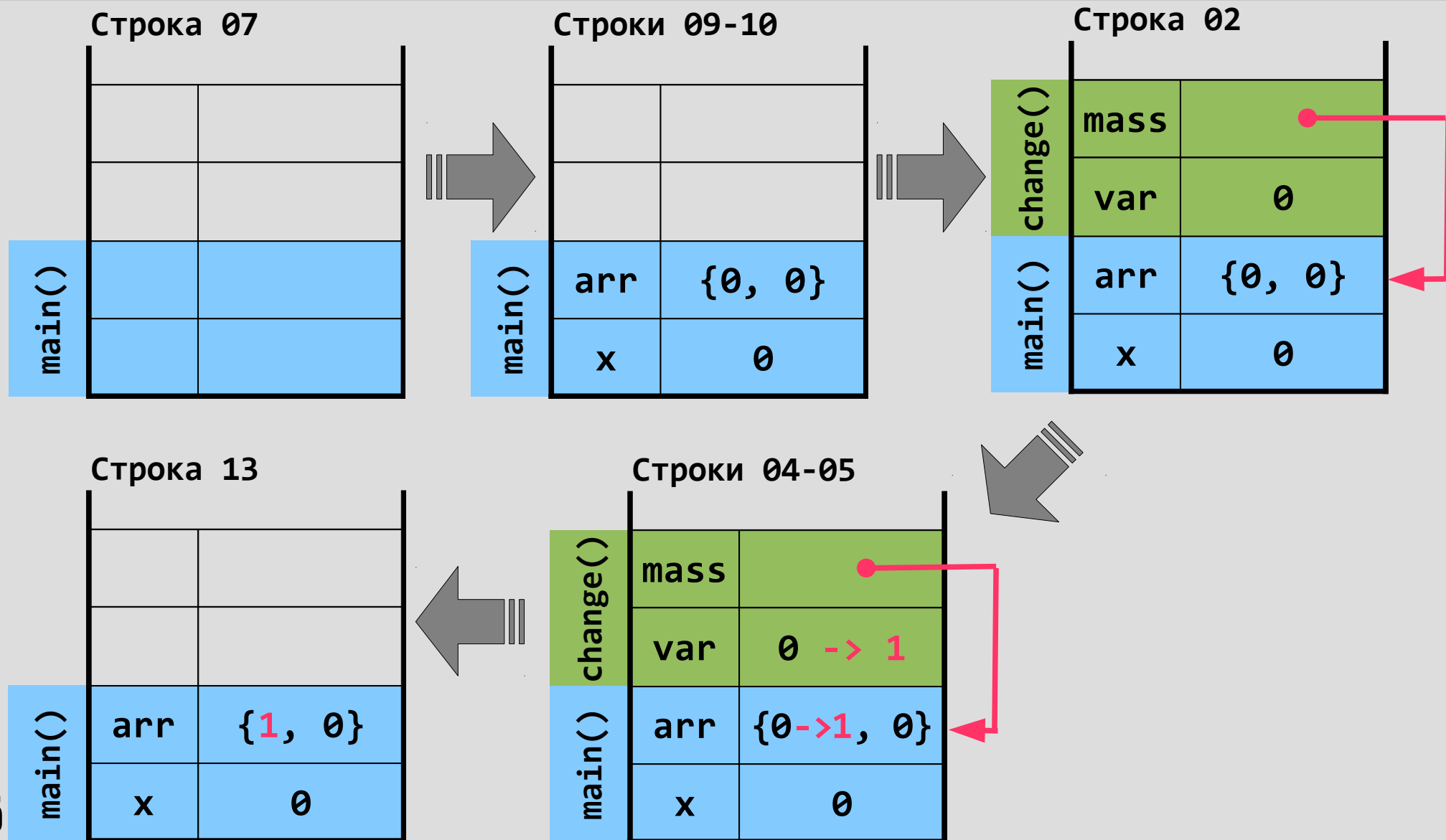
- Параметры простых типов данных передаются в функцию **по значению**, т.е. значения фактических параметров присваиваются (**копируются**) формальным параметрам функции
- Изменения формальных параметров внутри функции **не отражаются** на фактических параметрах (т.е. обратного копирования не происходит)
- Исключением из этих правил являются **массивы**, для которых копия **не** создается и функция может **изменить** исходный массив

Задание

Определите значения переменных `x` и `arr` после вызова функции `change()`

```
01 // Формальные параметры var и mass
02 void change( int var, int mass[ ] )
03 {
04     var++;
05     mass[0]++;
06 }
07 void main()
08 {
09     int x = 0;
10     int arr[2] = {0};
11     change(x, arr); // пытаемся изменить значения
12                     // фактических параметров x и arr
13 }
```

Передача параметров в функцию (трасса вычислений)



Передача одномерного массива в функцию

```
// Объявление функции, в которую передается  
// одномерный массив  
void handle_array( int arr[3] );    // прототип  
void handle_array( int arr[] );    // ... или  
void handle_array( int *arr );    // ... или
```

```
int a[3] = {1, 2, 3} ;
```

```
// Вызов функции, в которую передается одномерный  
// массив  
handle_array( a );
```

Передача многомерного массива в функцию

```
// Объявление функции, в которую передается
// матрица
void handle_matrix( int matr[2][3] );    // прототип
void handle_matrix( int matr[][3] );    // ... или

int b[2][3] = { {1, 2, 3} , {2, 3, 4} };

// Вызов функции, в которую передается матрица
handle_matrix( b );

// Вызов функции, в которую передается одномерный
// массив, который является подмассивом матрицы
handle_array( b[0] );
```

Это надо помнить!

- При вызове функции **круглые скобки** указывать обязательно даже если функция не имеет параметров. По наличию круглых скобок язык Си отличает функцию от переменной
- Если функция пытается изменять значения своих параметров, то эти изменения происходят с **копиями** параметров, созданными для функции, и **не** сохраняются в вызывающей функции
- Если в функцию передается **массив**, то его копия не создается, а все изменения **вносятся** в переданный массив

Ошибки при компиляции для VS 2008 Russian

- **<имя функции>**: идентификатор не найден - отсутствует объявление (прототип) функции (в случае системной функции не подключен необходимый h-файл)
- **ссылка на неразрешенный внешний символ "<прототип функции>" в функции ...** - отсутствует описание (тело) функции
- **<имя параметра функции>**: необъявленный идентификатор - в заголовке функции не указано имя ее параметра

Ошибки при компиляции для VS 2008 Russian

- **<имя функции>**: должна возвращать значение — функция имеет тип возвращаемого значения, однако не содержит оператора **return**, который мог бы его вернуть (или оператор **return** пустой)
- **return**: невозможно преобразовать '**<тип в операторе return>**' в '**<необходимый функции тип>**'
 - в операторе **return** указано значение, не подходящее по типу к возвращаемому значению функции

Ошибки при компиляции для VS 2008 Russian

- **<имя функции>: функция не принимает <количество> аргументов** — неверное количество аргументов при вызове функции
- **<имя функции>: невозможно преобразовать параметр <номер параметра> из '<тип при вызове>' в '<необходимый функции тип>'** - при вызове функции указанный параметр имеет неверный тип
- **<имя функции>: функция типа 'void', возвращающая значение** — в функции типа **void** в операторе **return** указано возвращаемое значение, хотя она не должна ничего возвращать

Ошибки при компиляции для VS 2005 English

- '**<имя функции>**': **identifier not found** - отсутствует объявление (прототип) функции (в случае системной функции не подключен необходимый h-файл)
- **unresolved external symbol <прототип функции>** - отсутствует описание (тело) функции
- '**<имя параметра функции>**': **undeclared identifier** - в заголовке функции не указано имя ее параметра

Ошибки при компиляции для VS 2005 English

- '**<имя функции>**': **must return value** — функция имеет тип возвращаемого значения, однако не содержит оператора **return**, который мог бы его вернуть (или оператор **return** пустой)
- '**return**': **cannot convert from '<тип в операторе return>' to '<необходимый функции тип>'** - в операторе **return** указано значение, не подходящее по типу к возвращаемому значению функции

Ошибки при компиляции для VS 2005 English

- '**<имя функции>**': **function does not take <количество> arguments** — неверное количество аргументов при вызове функции
- '**<имя функции>**': **cannot convert parameter <номер параметра> from '<тип при вызове>' to '<необходимый функции тип>'** - при вызове функции указанный параметр имеет неверный тип
- '**<имя функции>**': **void function returning a value** — в функции типа **void** в операторе **return** указано возвращаемое значение, хотя она не должна ничего возвращать